
Grupo Acessibilidade Arquitetura de software

**Acessibilidade Já
Software Architecture Document**

Versão <1.1>

Revision History

Date	Version	Description	Author
20/06/26	1.0	Primeira versão do DAS	Pedro Fonseca Cruz
20/06/26	1.1	Segunda versão do DAS	Fábio Santos Araújo

Table of Contents

1.	Introduction	2
1.1	Purpose	2
1.2	Scope	2
1.3	Definitions, Acronyms, and Abbreviations	2
1.4	References	2
1.5	Overview	2
2.	Architectural Representation	2
3.	Architectural Goals and Constraints	2
4.	Use-Case View	2
4.1	Use-Case Realizations	2
5.	Logical View	2
5.1	Overview	2
5.2	Architecturally Significant Design Packages	2
6.	Process View	2
7.	Deployment View	2
8.	Implementation View	2
8.1	Overview	2
8.2	Layers	2
9.	Data View (optional)	2
10.	Size and Performance	2
11.	Quality	2

Software Architecture Document

1. Introduction

O Acessibilidade Já visa adicionar várias funcionalidades de acessibilidade de forma rápida e fácil para os desenvolvedores ReactJS. Foi produzido um widget que desenvolvedores podem facilmente adicionar ao seu site. O widget manipula o DOM e modifica o estilo do site para adequar a vários tipos de usuários.

1.1 Purpose

Este documento tem como objetivo registrar a arquitetura do Acessibilidade Já, descrevendo os principais componentes, decisões arquiteturais, restrições, padrões de projeto, fluxos de execução e responsabilidades internas do sistema.

1.2 Scope

O escopo deste documento contempla a arquitetura do widget de acessibilidade implementado em JavaScript, sua página de demonstração e sua documentação associada. São descritas as visões de casos de uso, lógica, processo, implantação, implementação e dados, além de atributos de qualidade como extensibilidade, portabilidade, acessibilidade, manutenibilidade e desempenho.

Não faz parte do escopo deste DAS a modelagem completa de um back-end, pois o widget funciona majoritariamente no navegador e não depende de banco de dados próprio para executar suas funcionalidades centrais.

1.3 Definitions, Acronyms, and Abbreviations

DOM (Document Object Model): Estrutura em árvore que representa a página no navegador e permite que scripts modifiquem dinamicamente seu conteúdo, estilo e comportamento.

Widget: Componente de interface reutilizável que pode ser incorporado a uma aplicação maior.

JavaScript: Linguagem de programação utilizada no navegador para implementar interatividade, manipulação do DOM e integração com APIs web.

ReactJS: Biblioteca JavaScript para construção de interfaces de usuário. O projeto possui uma estrutura relacionada ao React/Next.js, embora o widget principal seja standalone.

GoF (Gang of Four): Conjunto de padrões de projeto descritos por Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides.

DAS (Documento de Arquitetura de Software): Documento que descreve a arquitetura do sistema, suas decisões de projeto, componentes e relacionamentos.

WCAG (Web Content Accessibility Guidelines): Diretrizes internacionais que definem boas práticas para tornar conteúdos web mais acessíveis.

API pública: Conjunto de funções disponibilizadas pelo widget por meio de `window.AcessibilidadeJa`, permitindo a interação controlada com o componente.

1.4 References

SERRANO, Milene. **Aula Arquitetura de Software – Visão Geral**. Universidade de Brasília, 2026.

PERRY, D. E.; WOLF, A. L. **Foundations for the Study of Software Architecture**. SIGSOFT Software Engineering Notes, 1992.

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. **Design Patterns: Elements of Reusable Object-Oriented Software**. Addison-Wesley, 1994.

W3C. **Web Content Accessibility Guidelines (WCAG) 2.2**.

Acessibilidade Já	Versão: 1.0
Software Architecture Document	Date: 20/06/26
<document identifier>	

1.5 Overview

O restante deste documento está organizado em visões arquiteturais. A seção de representação arquitetural explica quais visões são usadas. Em seguida, são apresentados objetivos e restrições que influenciam a arquitetura. A visão de casos de uso descreve os atores e interações principais. A visão lógica detalha os componentes internos e padrões de projeto. A visão de processo descreve o fluxo de execução no navegador. A visão de implantação mostra como o widget é distribuído e executado. A visão de implementação relaciona a arquitetura com os arquivos do projeto. Por fim, são apresentadas a visão de dados, considerações de tamanho e desempenho, e atributos de qualidade.

2. Architectural Representation

A arquitetura do Acessibilidade Já é representada por um recorte do modelo 4+1, adaptado ao contexto de um widget executado no navegador. As visões consideradas são:

Visão de Casos de Uso: descreve as interações entre desenvolvedor integrador, usuário final e serviços externos.

Visão Lógica: apresenta os principais componentes internos, responsabilidades e padrões de projeto.

Visão de Processo: descreve o fluxo de inicialização, interação, aplicação de comandos e atualização do DOM.

Visão de Implantação: mostra como o widget é disponibilizado como script estático e executado no navegador da página hospedeira.

Visão de Implementação: relaciona camadas, arquivos e módulos relevantes.

Essa representação foi escolhida porque permite documentar o sistema em diferentes níveis de abstração, evitando que a arquitetura fique limitada a um único diagrama ou a uma única perspectiva.

3. Architectural Goals and Constraints

A principal meta arquitetural do AcessibilidadeJá é permitir que desenvolvedores adicionem recursos de acessibilidade a páginas web de forma simples, reutilizável e pouco intrusiva. Para isso, o widget foi projetado como um script JavaScript standalone, capaz de executar sem depender diretamente do framework da aplicação hospedeira.

As principais restrições e objetivos arquiteturais são:

Facilidade de integração: o widget deve poder ser incluído por meio de uma tag `<script>`.

Baixo acoplamento: a implementação deve interferir o mínimo possível na estrutura da página hospedeira.

Portabilidade: o widget deve funcionar em diferentes páginas web modernas.

Acessibilidade: os controles devem usar atributos ARIA, foco visível e suporte a navegação por teclado.

Extensibilidade: novas funcionalidades devem poder ser adicionadas sem reescrever toda a estrutura.

Persistência local: preferências do usuário devem ser mantidas entre acessos por meio de armazenamento no navegador.

Prevenção de duplicidade: o widget não deve ser inicializado mais de uma vez na mesma página.

Evidência de padrões arquiteturais: a implementação deve demonstrar padrões GoF e decisões arquiteturais coerentes com o escopo da disciplina.

Essas restrições influenciam diretamente o uso de padrões como Singleton, Factory, Builder, Command, Decorator, Prototype e Facade.

4. Use-Case View

O diagrama de casos de uso representa as interações entre os atores — Desenvolvedor e Usuário — e o sistema Acessibilidade Já. O Desenvolvedor é responsável por configurar e integrar o widget ao site, enquanto o usuário interage com as funcionalidades de acessibilidade disponíveis, como filtro claro/escuro, tradutor, lupa, ajuste de fonte e contraste. O caso de uso central Abrir menu de acessibilidade é estendido por praticamente todas as funcionalidades, evidenciando que o menu é o ponto de entrada obrigatório para o sistema. Diagramas de casos de uso são fundamentais no desenvolvimento de software pois permitem comunicar, de forma clara e independente de tecnologia, o que o sistema deve fazer e para quem — facilitando o alinhamento entre desenvolvedores, designers e partes interessadas antes mesmo de qualquer linha de código ser escrita. Eles também servem como base para a definição de requisitos funcionais e para a criação de cenários de teste.

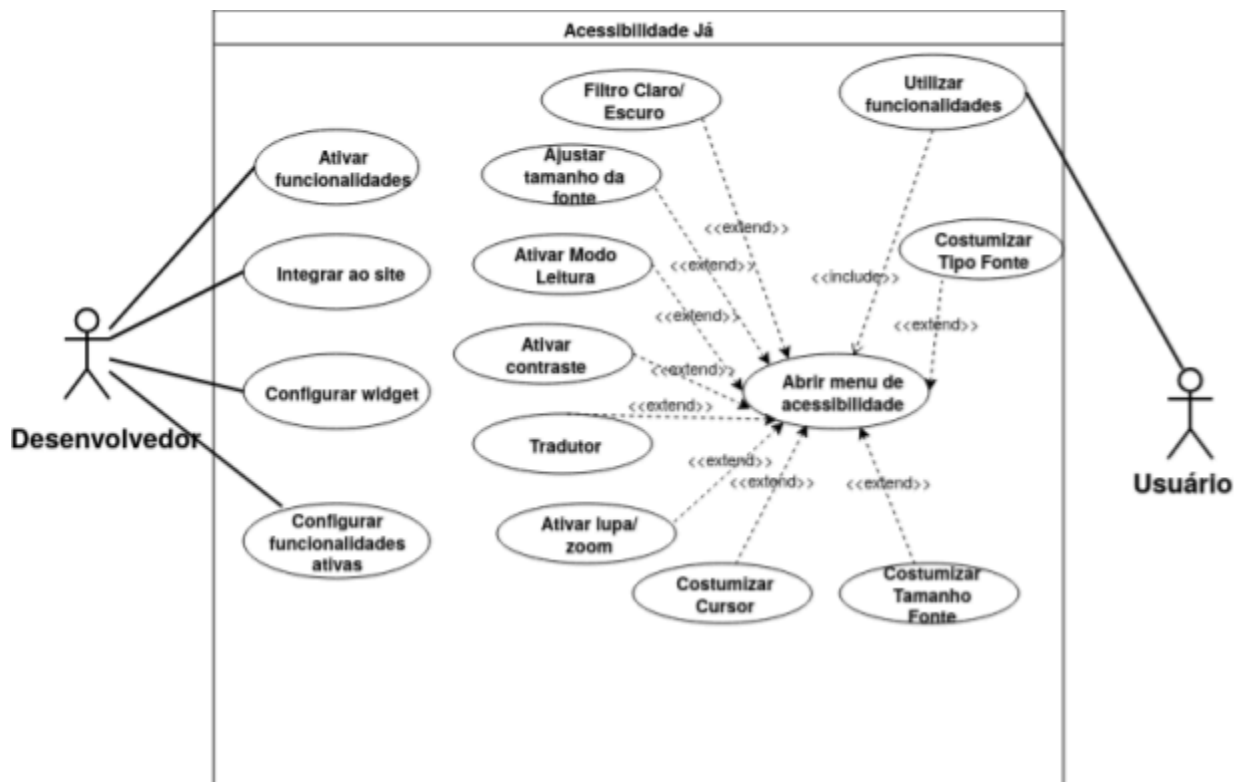
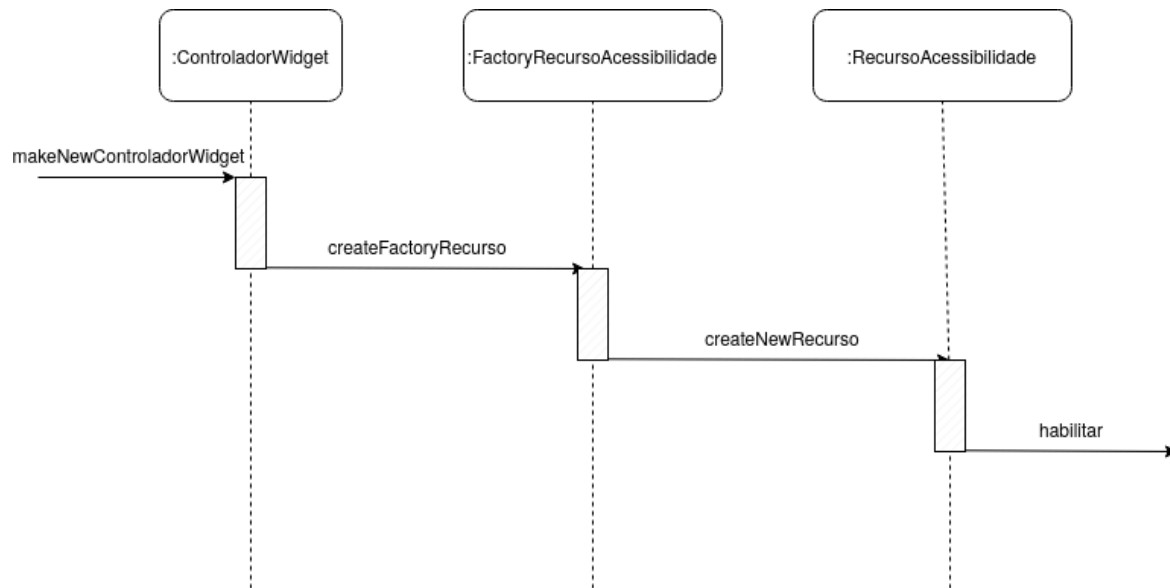


Diagrama de casos de uso UML

4.1 Use-Case Realizations

Para todas as funcionalidades a realização da funcionalidade é a mesma em um nível meta. O objeto da classe ControladorWidget chama o FactoryRecursoAcessibilidade que cria um objeto que herda de Recurso Acessibilidade que tem a função habilitar. A qual habilita a funcionalidade que a classe é responsável.



5. Logical View

A visão lógica descreve a decomposição interna do widget e suas principais responsabilidades. O **AcessibilidadeJá** é organizado em torno de componentes que cuidam da inicialização, configuração, criação de interface, manipulação de eventos, execução de comandos, aplicação de efeitos e exposição de uma API pública. Os elementos logicamente significativos são:

WidgetSingleton: impede múltiplas inicializações do widget na mesma página.

OptionPrototype: define uma estrutura base para opções de acessibilidade.

TOGGLE_OPTIONS: catálogo declarativo das funcionalidades disponíveis no menu.

IconFactory: centraliza a criação de ícones SVG usados pela interface.

WidgetElementFactory: cria elementos DOM do widget, como painel, botões, seletor de idioma e skip link.

WidgetBuilder: monta o widget em etapas encadeadas.

Command e CommandInvoker: encapsulam ações do usuário e permitem histórico de undo/redo.

AccessibilityEffect e Decorators: aplicam efeitos de acessibilidade de forma combinável.

createDomFacade: encapsula operações complexas de foco, atalhos e navegação por teclado.

5.1 Overview

Este projeto se propõe ser um widget, ou seja ele deve ser capaz de se acoplar em qualquer site desenvolvido

5.2 Architecturally Significant Design Packages

O diagrama de pacotes representa a arquitetura do widget AcessibilidadeJa, organizado em camadas que refletem suas responsabilidades dentro do sistema.

No nível mais externo está o pacote Hospedeiro, que representa o site que incorpora o widget. A classe SiteHospedeiro importa diretamente o pacote WidgetAcessibilidade — o núcleo da aplicação — e também o pacote React, biblioteca de terceiros usada na construção da interface através de sua VirtualDOM.

Dentro de WidgetAcessibilidade, os pacotes estão organizados em três camadas funcionais. A camada de interface (User_Interface) contém o Menu e o IconeAcessibilidade (classe abstrata), responsáveis pela interação visual com o usuário. Abaixo dela, a camada de controle (Controle_Acessibilidade) concentra a lógica da aplicação: o ControladorAcessibilidade recebe os comandos da interface e gerencia o RecursoAcessibilidade (também abstrato), que por sua vez delega ao ManipuladorDom as alterações concretas no documento. Por fim, a camada de persistência (Persistencia) guarda o estado das preferências de acessibilidade do usuário, através do ControladorPersistencia, que pode utilizar tanto Cookie quanto LocalStorage como estratégias de armazenamento.

Externamente ao widget, o pacote Navegador representa as APIs nativas do browser (DOM, LocalStorage, Window), que são efetivamente consumidas tanto pelo ManipuladorDom quanto pelo ControladorPersistencia e pela VirtualDOM do React — evidenciando que, no fim, toda manipulação de interface e armazenamento converge para a mesma infraestrutura do navegador.

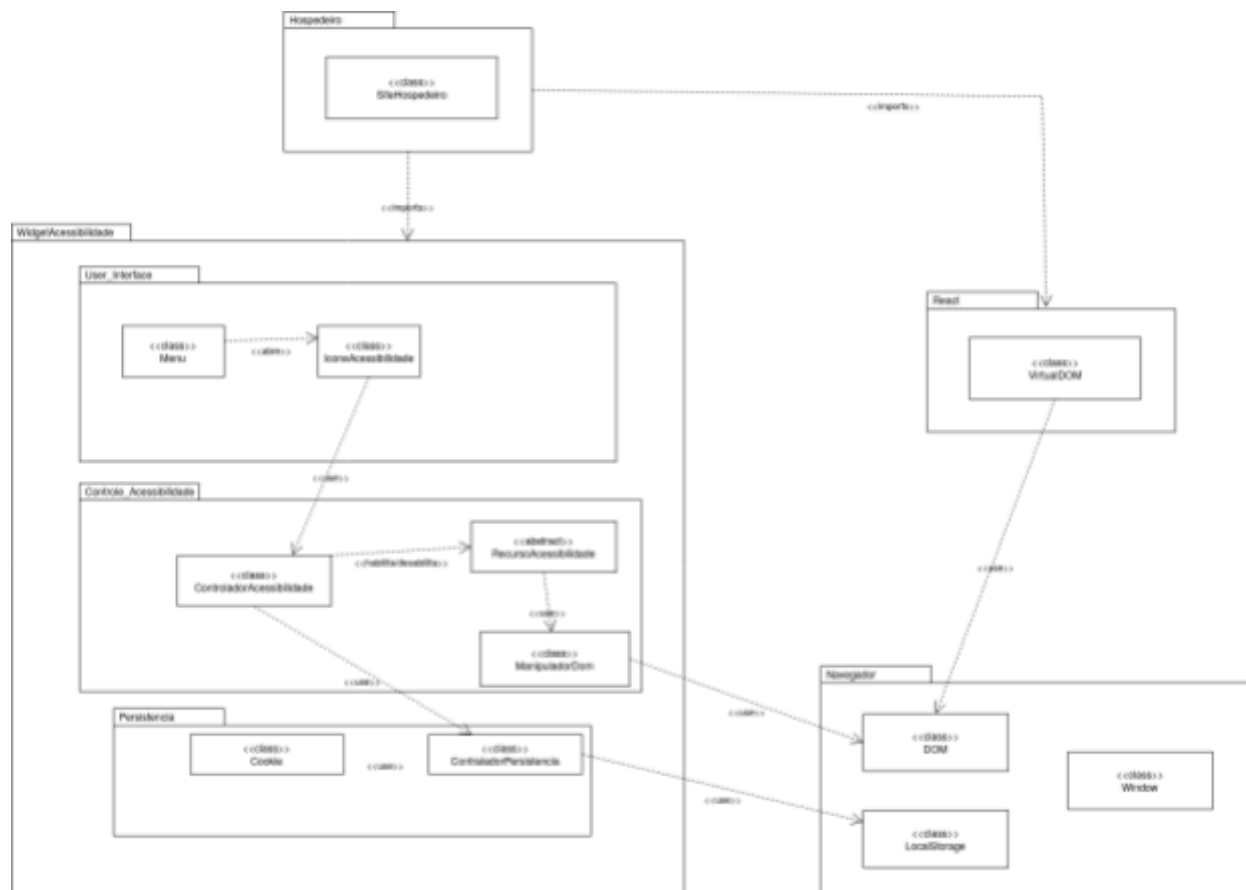


Diagrama de Pacotes UML

6. Process View

O AcessibilidadeJa executa dentro do processo do navegador, utilizando o ciclo de eventos JavaScript. Não há processos paralelos próprios nem serviços internos em execução contínua. A arquitetura depende de

eventos como clique, mudança de seleção, pressionamento de tecla e movimentação do mouse quando o guia de leitura está ativo.

O fluxo principal ocorre da seguinte forma:

5. A página hospedeira carrega o script do widget.
6. O widget verifica se já foi inicializado.
7. O estado salvo é recuperado do localStorage.
8. Os estilos e elementos do widget são injetados no DOM.
9. A função de aplicação inicial sincroniza interface, estado e efeitos.
10. O usuário abre o painel pelo botão flutuante.
11. Cada ação do usuário cria e executa um comando.
12. O estado atualizado é aplicado por meio dos Decorators.

7. Deployment View

O widget é implantado como um arquivo JavaScript estático, que pode ser servido junto com a aplicação hospedeira ou por um servidor estático. Na demonstração do projeto, a pasta `projetoconfgofs` usa Vite para servir a página local e carregar o script do widget.

Os principais nós de implantação são:

- Repositório GitHub: armazena código, documentação, issues e histórico de participação.
- GitPages/MkDocs: publica a documentação do projeto.
- Servidor estático ou Vite: disponibiliza os arquivos da demonstração.
- Navegador do usuário: executa o widget, manipula o DOM e armazena preferências locais.

8. Implementation View

A implementação do Acessibilidade Já é concentrada principalmente no arquivo `projetoconfgofs/public/widget/acessibilidadeja.js`. Esse arquivo contém a inicialização, estilos, criação da interface, comandos, aplicação de efeitos, persistência de estado, fachada DOM e API pública.

Também existe o arquivo `projetoconfgofs/public/widget/decorator-pattern.js`, que documenta de forma isolada a modelagem do padrão Decorator utilizado no widget. A demonstração local é servida a partir de `projetoconfgofs/index.html`, com arquivos auxiliares de site e estilos.

8.1 Overview

A implementação pode ser compreendida em camadas:

API pública: funções expostas em `window.AcessibilidadeJa`.

Eventos de interface: cliques, mudanças de seleção e eventos de teclado.

Comandos e estado: objetos que encapsulam alterações e controlam histórico.

Efeitos de acessibilidade: cadeia de Decorators que aplica classes, zoom, guia de leitura, tradução e persistência.

Fábricas e fachada DOM: criação de elementos e encapsulamento de detalhes do navegador.

Recursos do navegador: DOM, CSSOM, localStorage, sessionStorage e cookies.

8.2 Layers

Camada de API pública: Fornece métodos como `open`, `close`, `reset`, `setReadingMode`, `getState`, `undo`, `redo`, `canUndo`, `canRedo`, `getCommandHistory` e `getDecoratorChain`, permitindo que aplicações externas interajam com o widget de forma controlada.

Camada de interface: Contém os elementos visuais do sistema, incluindo o botão flutuante, painel lateral, overlay, botões de recursos, seletor de idioma, controle de zoom e botão de redefinição (reset).

Camada de comandos: Converte as ações realizadas pelo usuário em operações sobre o estado da aplicação. Essa camada reduz a duplicação de código e possibilita o gerenciamento do histórico de ações.

Camada de efeitos: Interpreta o estado atual da aplicação e aplica os recursos de acessibilidade ao DOM por meio da utilização de Decorators.

Camada de infraestrutura do navegador: Responsável pela integração com recursos do navegador, utilizando armazenamento local (Local Storage), cookies, scripts externos e manipulação direta do DOM.

9. Data View (optional)

O widget não possui banco de dados próprio. A persistência é local ao navegador.

10. Size and Performance

O widget foi projetado para ser leve e executado no navegador da página hospedeira. A maior parte dos recursos é aplicada por classes CSS no elemento `<html>`, evitando percorrer manualmente toda a árvore DOM em cada interação.

A função de aplicação reconstrói a cadeia de Decorators com base no estado atual e aplica apenas os efeitos necessários. O guia de leitura adiciona listener de `'mousemove'` somente enquanto está ativo, removendo-o quando a funcionalidade é desativada. O serviço externo de tradução é carregado apenas sob demanda, reduzindo o custo inicial de carregamento.

O histórico de comandos é limitado a 50 entradas, evitando crescimento indefinido em memória. Essas decisões contribuem para desempenho aceitável em páginas comuns, preservando a simplicidade da integração.

11. Quality

A arquitetura contribui para a qualidade do sistema em diferentes aspectos:

Acessibilidade: O painel usa atributos ARIA, estados aria-pressed, foco visível, trap de foco, skip link e suporte a navegação por teclado.

Extensibilidade: Novas opções podem ser adicionadas ao catálogo de toggles e conectadas a comandos ou Decorators específicos.

Manutenibilidade: O uso de padrões como Factory, Builder, Command, Decorator e Facade separa responsabilidades e torna o código mais compreensível.

Portabilidade: O widget é standalone e não depende diretamente do framework da página hospedeira.

Reutilização: fábricas, fachada DOM e Decorators podem ser reaproveitados em novas funcionalidades do widget.

Confiabilidade: há tratamento de erros ao acessar armazenamento local, cookies e operações de DOM. O MutationObserver remove recursos auxiliares quando o root do widget deixa de existir.

Segurança e privacidade: O widget armazena apenas preferências locais de acessibilidade. A funcionalidade de tradução depende de serviço externo e deve ser documentada como integração opcional.

